

LAPORAN PRAKTIKUM PEMROGRAMAN WEB
Modul 12: React Router, Autentikasi, dan REST API



Disusun Oleh:

Nama: Dryhus Dzacky Damingtyas.S

NIM: 707012530010

Kelas: D4SIKC-49-04

Mata Kuliah: Pemrograman Web

UNIVERSITAS TELKOM
FAKULTAS ILMU TERAPAN
JURUSAN S1 TERAPAN SISTEM INFORMASI KOTA CERDAS
TAHUN AJARAN 2026

A. Tujuan

Setelah mengikuti praktikum ini mahasiswa diharapkan dapat:

1. Memahami penggunaan React Router untuk membuat aplikasi single page application dengan beberapa halaman.
2. Memahami konsep autentikasi pada aplikasi React menggunakan token yang disimpan di localStorage.
3. Mampu membuat AuthContext untuk mengelola data user, token, login, logout, dan validasi sesi.
4. Mampu membuat protected route agar halaman tertentu hanya dapat diakses oleh user yang sudah login.
5. Mampu mengintegrasikan React dengan REST API CodeIgniter 3 menggunakan axios.
6. Mampu menampilkan daftar post dan detail post dari backend melalui request API.
7. Mampu menggunakan Tailwind CSS untuk membuat antarmuka login, navbar, card, footer, dan halaman artikel yang responsif.

B. Alat dan Bahan

Untuk menjalankan praktikum ini diperlukan komputer atau laptop dengan sistem operasi Windows, Linux, atau macOS, minimal prosesor 1 GHz, RAM 2GB (disarankan 4GB atau lebih), serta browser modern seperti Google Chrome, Mozilla Firefox, atau Microsoft Edge. Perangkat lunak yang digunakan meliputi:

- Code editor (Visual Studio Code, Sublime Text, atau sejenisnya)
- Node.js dan npm
- React JS dan Vite
- React Router DOM
- Axios
- Tailwind CSS
- XAMPP dan backend CodeIgniter 3 Pekan-09
- Web browser

Bahan yang dibutuhkan:

- Folder project React pada Pekan-12
- File JSX untuk halaman, komponen, context, dan service API
- Endpoint REST API CodeIgniter 3 pada folder Pekan-09
- Token autentikasi dari endpoint login API

C. Langkah-Langkah Praktikum

1. Buka folder `Pekan-12` pada direktori XAMPP htdocs.
2. Pastikan dependency project terpasang menggunakan perintah `npm install`.
3. Tambahkan Tailwind CSS melalui file `tailwind.config.js`, `postcss.config.js`, dan directive Tailwind pada `src/index.css`.
4. Buat service API pada `src/services/api.js` menggunakan axios dengan base URL backend CodeIgniter 3 Pekan-09.
5. Tambahkan interceptor request untuk mengirim token JWT melalui header Authorization Bearer.

6. Tambahkan interceptor response untuk menghapus sesi dan mengarahkan user ke halaman login ketika token tidak valid.
7. Buat `AuthContext.jsx` untuk menyimpan user, token, loading, login, logout, dan validasi token lama.
8. Buat `ProtectedRoute.jsx` untuk melindungi halaman post dari user yang belum login.
9. Atur route pada `App.jsx`, meliputi `/login`, `/posts`, `/posts/:id`, redirect root ke `/posts`, dan fallback route.
10. Buat halaman `Login.jsx` untuk menerima email dan password, lalu memanggil fungsi login dari AuthContext.
11. Buat halaman `PostList.jsx` untuk mengambil daftar artikel dari endpoint `api/post` dan menampilkan loading, error, empty state, serta daftar post.
12. Buat `PostCard.jsx` untuk menampilkan judul, ringkasan artikel, author, tanggal, dan link menuju detail post.
13. Buat halaman `PostDetail.jsx` untuk mengambil detail artikel berdasarkan ID dari URL `/posts/:id`.
14. Buat komponen pendukung `Navbar.jsx`, `Footer.jsx`, dan `DefaultAvatar.jsx` agar tampilan halaman konsisten.
15. Jalankan aplikasi menggunakan perintah `npm run dev`.
16. Uji halaman login, validasi protected route, daftar post, detail post, logout, dan pesan error ketika backend belum berjalan.

D. Hasil dan Dokumentasi Program

```
{
  "name": "pekan-12",
  "private": true,
  "version": "0.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "lint": "eslint .",
    "preview": "vite preview"
  },
  "dependencies": {
    "axios": "^1.16.1",
    "react": "^19.2.6",
    "react-dom": "^19.2.6",
    "react-router-dom": "^7.15.1"
  },
  "devDependencies": {
    "@eslint/js": "^10.0.1",
    "@types/react": "^19.2.14",
    "@types/react-dom": "^19.2.3",
    "@vitejs/plugin-react": "^6.0.1",
    "autoprefixer": "^10.4.24",
    "eslint": "^10.3.0",
    "eslint-plugin-react-hooks": "^7.1.1",
    "eslint-plugin-react-refresh": "^0.5.2",
    "globals": "^17.6.0",
    "postcss": "^8.5.6",
    "tailwindcss": "^3.4.19",
    "vite": "^8.0.12"
  }
}
```

Ini adalah package.json

```
/** @type {import('tailwindcss').Config} */
export default {
  // Daftar file yang dipindai Tailwind agar class utility yang dipakai ikut dibuat.
  content: ['./index.html', './src/**/*.{js,ts,jsx,tsx}'],
  // Mode dark disiapkan sesuai modul, walaupun tampilan utama praktikum memakai light mode.
}
```

```

darkMode: 'class',
theme: {
  extend: {
    colors: {
      // Palet primary mengikuti instruksi modul praktikum Pekan 12.
      primary: {
        50: '#eff6ff',
        100: '#dbeafe',
        200: '#bfdbfe',
        300: '#93c5fd',
        400: '#60a5fa',
        500: '#3b82f6',
        600: '#2563eb',
        700: '#1d4ed8',
        800: '#1e40af',
        900: '#1e3a8a',
      },
    },
  },
},
// Plugin kosong karena modul hanya membutuhkan utility bawaan Tailwind.
plugins: [],
}

```

Ini adalah tailwind.config.js

```

// StrictMode membantu menampilkan peringatan React saat proses development.
import { StrictMode } from 'react'
// createRoot adalah API React DOM modern untuk merender aplikasi ke elemen HTML.
import { createRoot } from 'react-dom/client'
// File CSS utama berisi Tailwind dan style global aplikasi.
import './index.css'
// Komponen App menjadi pintu masuk seluruh routing aplikasi.
import App from './App.jsx'

// Menghubungkan aplikasi React ke elemen <div id="root"> di index.html.
createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)

```

Ini adalah src/main.jsx

```

// Komponen routing dari React Router untuk mengatur perpindahan halaman SPA.
import { BrowserRouter as Router, Navigate, Route, Routes } from 'react-router-dom'
// AuthProvider menyediakan data login untuk seluruh halaman.
import { AuthProvider } from './context/AuthContext'
// ProtectedRoute menjaga halaman post agar hanya bisa dibuka setelah login.
import ProtectedRoute from './components/ProtectedRoute'
// Halaman login untuk mendapatkan token dari API CI3.
import Login from './pages/Login'
// Halaman daftar post dari backend CodeIgniter Pekan-09.
import PostList from './pages/PostList'
// Halaman detail post berdasarkan ID.
import PostDetail from './pages/PostDetail'

function App() {
  return (
    // Provider diletakkan paling luar agar semua route dapat membaca status autentikasi.
    <AuthProvider>
      <Router>
        <Routes>
          { /* Route login terbuka tanpa token. */ }
          <Route path="/login" element={<Login />} />

          { /* Route daftar post dilindungi token JWT dari API. */ }
          <Route
            path="/posts"
            element={
              <ProtectedRoute>
                <PostList />
              </ProtectedRoute>
            }
          />
        </Routes>
      </Router>
    </AuthProvider>
  )
}

```

```

    { /* Route detail post mengambil ID dari URL /posts/:id. */}
    <Route
      path="/posts/:id"
      element={
        <ProtectedRoute>
          <PostDetail />
        </ProtectedRoute>
      }
    />

    { /* Root aplikasi langsung diarahkan ke halaman posts sesuai modul. */}
    <Route path="/" element={<Navigate to="/posts" replace />} />

    { /* Route yang tidak dikenal diarahkan kembali ke posts. */}
    <Route path="*" element={<Navigate to="/posts" replace />} />
  </Routes>
</Router>
</AuthProvider>
)
}

export default App

```

Ini adalah src/App.jsx

```

// Axios dipakai sebagai HTTP client untuk mengakses REST API CodeIgniter.
import axios from 'axios'

// Base URL diarahkan langsung ke project CI3 Pekan-09 yang berada di dalam htdocs XAMPP.
const API_URL = 'http://localhost/Pemrograman%20Web/WebPro4904/Pekan-09/ci3_project/index.php/api'

// Instance axios pusat agar konfigurasi header dan interceptor tidak diulang di setiap request.
const api = axios.create({
  baseURL: API_URL,
  headers: {
    'Content-Type': 'application/json',
    Accept: 'application/json',
  },
})

// Interceptor request menempelkan token JWT ke header Authorization bila user sudah login.
api.interceptors.request.use(
  (config) => {
    // Token disimpan di localStorage oleh AuthContext setelah proses login sukses.
    const token = localStorage.getItem('token')

    // Header Bearer dibaca oleh library Jwt.php pada backend CodeIgniter.
    if (token) {
      config.headers.Authorization = `Bearer ${token}`
    }

    // Config wajib dikembalikan agar axios dapat melanjutkan request.
    return config
  },
  (error) => Promise.reject(error),
)

// Interceptor response menangani token yang sudah invalid atau expired.
api.interceptors.response.use(
  (response) => response,
  (error) => {
    // Jika backend mengembalikan 401 di luar halaman auth, sesi lokal dihapus.
    if (error.response(c).status === 401) {
      const requestUrl = error.config(c).url || ''

      // Login/register tidak dipaksa redirect agar pesan error tetap bisa tampil di form.
      if (!requestUrl.includes('/login') && !requestUrl.includes('/register')) {
        localStorage.removeItem('token')
      }
    }
  }
)

```

```

        localStorage.removeItem('user')
        // Redirect ke halaman login React saat token sudah tidak diterima backend.
        window.location.href = '/login'
    }
}

// Error tetap diteruskan supaya halaman dapat menampilkan pesan yang sesuai.
return Promise.reject(error)
},
)

// API autentikasi mengikuti route yang sudah ada di Pekan-09: api/auth/login, register,
logout, me.
export const authAPI = {
  login: (email, password) => api.post('/auth/login', { email, password }),
  register: (name, email, password) => api.post('/auth/register', { name, email, password }),
  logout: () => api.post('/auth/logout'),
  me: () => api.get('/auth/me'),
}

// API post mengikuti route Pekan-09: api/post untuk list dan api/post/:id untuk detail.
export const postsAPI = {
  getAll: (page = 1, perPage = 10) => api.get(`/post(c)page=${page}&per_page=${perPage}`),
  getById: (id) => api.get(`/post/${id}`),
}

```

```

// Export default dipakai bila suatu saat butuh request custom di luar authAPI/postsAPI.
export default api

```

Ini adalah src/services/api.js

```

/* eslint-disable react-refresh/only-export-components */
// Context React dipakai agar data login bisa diakses dari banyak komponen tanpa prop
drilling.
import { createContext, useContext, useEffect, useState } from 'react'
// authAPI berisi function request ke endpoint auth CodeIgniter.
import { authAPI } from '../services/api'

// AuthContext menyimpan user, token, loading, login, dan logout.
const AuthContext = createContext(null)

// AuthProvider membungkus aplikasi dan menjadi sumber data autentikasi.
export const AuthProvider = ({ children }) => {
  // User diambil dari localStorage agar sesi tetap ada saat browser direfresh.
  const [user, setUser] = useState(() => {
    // Data user tersimpan sebagai JSON string dari proses login sebelumnya.
    const savedUser = localStorage.getItem('user')
    // Jika belum ada user tersimpan, nilai awal dibuat null.
    return savedUser ? JSON.parse(savedUser) : null
  })
  // Token awal dibaca dari localStorage karena backend memakai JWT Bearer.
  const [token, setToken] = useState(() => localStorage.getItem('token'))
  // Loading mencegah halaman posts dibuka sebelum token lama selesai divalidasi ke backend.
  const [loading, setLoading] = useState(true)

  // Effect ini memvalidasi token lama agar token rusak/expired tidak membuat user terjebak di
  // posts.
  useEffect(() => {
    // Function async kecil dibuat di dalam effect untuk memanggil endpoint /auth/me.
    const validateSavedToken = async () => {
      // Jika tidak ada token, pengecekan selesai dan user tetap diarahkan ke login.
      if (!token) {
        setLoading(false)
        return
      }

      try {
        // Endpoint me memastikan token di localStorage masih diterima oleh backend CI3.
        const response = await authAPI.me()
        const payload = response.data(c).data

        // Data user dari backend dipakai untuk menyegarkan localStorage.

```

```

        if (payload) {
            const userData = {
                id: payload.id,
                name: payload.username,
                username: payload.username,
                email: payload.email,
                role: payload.role,
            }

            localStorage.setItem('user', JSON.stringify(userData))
            setUser(userData)
        }
    } catch (error) {
        // Token lama yang gagal diverifikasi dihapus supaya user bisa login ulang dengan
token baru.
        console.error('Token validation error:', error)
        localStorage.removeItem('token')
        localStorage.removeItem('user')
        setToken(null)
        setUser(null)
    } finally {
        // Setelah validasi selesai, ProtectedRoute boleh menentukan halaman berikutnya.
        setLoading(false)
    }
}

validateSavedToken()
}, [token])

// Function login dipanggil dari halaman Login.jsx.
const login = async (email, password) => {
    try {
        // Token lama dibersihkan dulu agar request berikutnya tidak memakai sesi yang rusak.
        localStorage.removeItem('token')
        localStorage.removeItem('user')
        // Backend CI3 mengembalikan struktur { status, message, data: { access_token, ...user }
    }.

    const response = await authAPI.login(email, password)
    const payload = response.data(c).data

    // Jika token tidak ada, response dianggap tidak valid untuk login.
    if (!payload(c).access_token) {
        return { success: false, message: response.data(c).message || 'Token tidak ditemukan
dari server.' }
    }

    // User disusun dari data backend tanpa menyimpan token di dalam object user.
    const userData = {
        id: payload.id,
        name: payload.username,
        username: payload.username,
        email: payload.email,
        role: payload.role,
    }

    // Token dan user disimpan agar sesi tetap aktif setelah refresh.
    localStorage.setItem('token', payload.access_token)
    localStorage.setItem('user', JSON.stringify(userData))
    setToken(payload.access_token)
    setUser(userData)

    // Return sukses dipakai oleh Login.jsx untuk redirect ke /posts.
    return { success: true }
} catch (error) {
    // Pesan error diambil dari backend bila ada, lalu diberikan ke halaman login.
    const message = error.response(c).data(c).message || 'Login gagal. Periksa email dan
password.'
    return { success: false, message }
}
}

// Function logout menghapus sesi di client dan mencoba memanggil endpoint logout backend.

```

```

const logout = async () => {
  try {
    // Endpoint logout CI3 tidak menyimpan blacklist token, tetapi tetap dipanggil sesuai
    modul.
    await authAPI.logout()
  } catch (error) {
    // Error logout tidak menghentikan pembersihan sesi lokal.
    console.error('Logout error:', error)
  } finally {
    // Data lokal dihapus agar ProtectedRoute mengembalikan user ke login.
    localStorage.removeItem('token')
    localStorage.removeItem('user')
    setToken(null)
    setUser(null)
  }
}

// Object value ini yang dibaca melalui hook useAuth().
const value = {
  user,
  token,
  loading,
  isAuthenticated: !!token,
  login,
  logout,
}

// Provider mengirim value ke seluruh child di dalam App.jsx.
return <AuthContext.Provider value={value}>{children}</AuthContext.Provider>
}

// Hook kecil agar komponen bisa membaca AuthContext dengan syntax yang ringkas.
export const useAuth = () => {
  // useContext mengambil value terbaru dari AuthProvider.
  const context = useContext(AuthContext)

  // Guard ini membantu menemukan error jika hook dipakai di luar AuthProvider.
  if (!context) {
    throw new Error('useAuth harus digunakan di dalam AuthProvider')
  }

  // Context dikembalikan ke komponen pemanggil.
  return context
}

```

Ini adalah src/context/AuthContext.jsx

```

// Navigate dipakai untuk memindahkan user yang belum login ke halaman /login.
import { Navigate } from 'react-router-dom'
// useAuth membaca status autentikasi dari AuthContext.
import { useAuth } from '../context/AuthContext'

// ProtectedRoute membungkus halaman yang membutuhkan token.
const ProtectedRoute = ({ children }) => {
  // isAuthenticated bernilai true bila token tersedia di state/localStorage.
  const { isAuthenticated, loading } = useAuth()

  // Saat AuthContext masih mengecek localStorage, tampilkan spinner sederhana.
  if (loading) {
    return (
      <div className="min-h-screen bg-gray-100 flex flex-col items-center justify-center gap-4">
        <div className="h-12 w-12 animate-spin rounded-full border-4 border-blue-100 border-b-blue-600"></div>
        <p className="text-sm font-medium text-gray-600">Loading...</p>
      </div>
    )
  }

  // Jika token tidak ada, user diarahkan ke login.
  if (!isAuthenticated) {
    return <Navigate to="/login" replace />
  }
}

```



```
// Export default agar komponen mudah diimport di App.jsx.
export default ProtectedRoute
```

```
// useState dipakai untuk membuka dan menutup menu mobile.
import { useState } from 'react'
// Link menjaga navigasi tetap SPA tanpa reload halaman.
import { Link } from 'react-router-dom'
// useAuth menyediakan data user dan function logout.
import { useAuth } from '../context/AuthContext'
// Avatar default ditampilkan di samping nama user.
import DefaultAvatar from './DefaultAvatar'
```

```

        /* Area kanan berisi user login, logout, dan tombol menu mobile. */
        <div className="flex items-center space-x-4">
            <div className="hidden sm:flex items-center space-x-3">
                <DefaultAvatar className="w-8 h-8" />
                <span className="text-sm font-medium text-gray-900">{user(c).name ||
user(c).username || 'User'}</span>
            </div>

```

```

        <button
          type="button"
          onClick={handleLogout}
          className="text-sm text-white bg-red-500 hover:bg-red-600 font-medium rounded-lg
px-4 py-2 transition-colors"
        >
          Logout
        </button>

        <button
          type="button"
          onClick={() => setMobileMenuOpen(!mobileMenuOpen)}
          className="md:hidden p-2 text-gray-600 hover:text-gray-900"
          aria-label="Buka menu"
        >
          <svg className="w-6 h-6" fill="none" stroke="currentColor" viewBox="0 0 24 24"
aria-hidden="true">
            <path strokeLinecap="round" strokeLinejoin="round" strokeWidth="2" d="M4
6h16M4 12h16M4 18h16"></path>
          </svg>
        </button>
      </div>
    </div>

    { /* Menu mobile hanya muncul saat tombol hamburger ditekan. */ }
    {mobileMenuOpen && (
      <div className="md:hidden pb-4 border-t border-gray-200 mt-2 pt-4">
        <div className="flex flex-col space-y-3">
          <Link to="/posts" className="text-blue-600 font-medium">
            Home
          </Link>
          <a href="#about" className="text-gray-600 hover:text-gray-900">
            About
          </a>
          <a href="#contact" className="text-gray-600 hover:text-gray-900">
            Contact
          </a>
        </div>
      </div>
    )}
  </div>
</nav>
)
}

```

```

// Export default agar Navbar dapat digunakan di halaman PostList dan PostDetail.
export default Navbar

```

Ini adalah src/components/Navbar.jsx

```

// Link dipakai untuk membuka detail post tanpa reload halaman.
import { Link } from 'react-router-dom'
// Avatar default menampilkan ikon author pada card.
import DefaultAvatar from './DefaultAvatar'

// PostCard menerima satu object post dari halaman PostList.
const PostCard = ({ post }) => {
  // Fungsi ini memotong artikel agar card tidak terlalu tinggi.
  const truncateText = (text, maxLength = 150) => {
    if (!text) return ''
    if (text.length <= maxLength) return text
    return `${text.substring(0, maxLength)}...`
  }

  // Fungsi tanggal menampilkan selisih hari seperti contoh modul.
  const formatDate = (dateString) => {
    if (!dateString) return 'Tanggal belum tersedia'

    // Date dibuat dari field created_at yang dikirim tabel posts.
    const date = new Date(dateString)
    const now = new Date()
    const diffDays = Math.floor((now - date) / (1000 * 60 * 60 * 24))
  }
}

```

```

    if (Number.isNaN(diffDays)) return 'Tanggal belum tersedia'
    if (diffDays === 0) return 'Today'
    if (diffDays === 1) return 'Yesterday'
    return `${diffDays} days ago`
  }

  return (
    <article className="p-6 bg-white rounded-lg border border-gray-200 shadow-md hover:shadow-
lg transition-shadow">
      {/ * Header card berisi kategori dan tanggal post. */}
      <div className="flex justify-between items-center gap-4 mb-5 text-gray-500">
        <span className="bg-blue-100 text-blue-800 text-xs font-medium inline-flex items-
center px-2.5 py-0.5 rounded">
          <svg className="mr-1 w-3 h-3" fill="currentColor" viewBox="0 0 20 20" aria-
hidden="true">
            <path
              fillRule="evenodd"
              d="M2 5a2 2 0 012-2h8a2 2 0 012 2v10a2 2 0 00 2 2h4a2 2 0 01-2-2v5zm3
1h6v4H5V6zm6 6H5v2h6v-2z"
              clipRule="evenodd"
            ></path>
            <path d="M15 7h1a2 2 0 012 2v5.5a1.5 1.5 0 01-3 0V7z"></path>
          </svg>
          Article
        </span>

        <span className="text-sm flex items-center text-right">
          <svg className="w-4 h-4 mr-1 flex-shrink-0" fill="currentColor" viewBox="0 0 20 20"
aria-hidden="true">
            <path
              fillRule="evenodd"
              d="M6 2a1 1 0 00-1 1v1H4a2 2 0 00-2 2v10a2 2 0 00 2 2h12a2 2 0 00 2 2V6a2 2 0 00-
2-2h-1V3a1 1 0 010-2 0v1H7V3a1 1 0 00-1-1zm0 5a1 1 0 00 2 2h8a1 1 0 00-2 2h6z"
              clipRule="evenodd"
            ></path>
          </svg>
          {formatDate(post.created_at)}
        </span>
      </div>

      {/ * Judul post mengarah ke route detail berdasarkan ID. */}
      <h2 className="mb-2 text-2xl font-bold tracking-tight text-gray-900">
        <Link to={` /posts/${post.id}`} className="hover:text-blue-600 transition-colors">
          {post.title}
        </Link>
      </h2>

      {/ * Ringkasan artikel diambil dari field article milik tabel posts. */}
      <p className="mb-5 font-light text-gray-500">{truncateText(post.article)}</p>

      {/ * Footer card berisi author dan tombol read more. */}
      <div className="flex justify-between items-center gap-4">
        <div className="flex items-center space-x-3 min-w-0">
          <DefaultAvatar />
          <span className="font-medium text-gray-900 truncate">{post.author}</span>
        </div>

        <Link
          to={` /posts/${post.id}`}
          className="inline-flex items-center font-medium text-blue-600 hover:text-blue-500
transition-colors flex-shrink-0"
        >
          Read more
          <svg className="ml-2 w-4 h-4" fill="currentColor" viewBox="0 0 20 20" aria-
hidden="true">
            <path
              fillRule="evenodd"
              d="M10.293 3.293a1 1 0 011.414 016 6a1 1 0 010 1.4141-6 6a1 1 0 01-1.414-
1.414L14.586 11H3a1 1 0 00-2 2h11.5861-4.293-4.293a1 1 0 010-1.414z"
              clipRule="evenodd"
            ></path>
          </svg>
        </Link>
      </div>

```

```
// Export default agar PostList dapat merender banyak PostCard.  
export default PostCard
```

```
// DefaultAvatar menjadi gambar profil cadangan karena backend tidak menyediakan foto user.
const DefaultAvatar = ({ className = 'w-7 h-7' }) => (
  // Ukuran avatar dapat diubah lewat prop className dari komponen pemanggil.
  <div className={` ${className} rounded-full bg-gray-300 flex items-center justify-center flex-shrink-0`}>
    {/* Icon user sederhana dibuat dengan SVG agar tidak perlu menambah library icon. */}
    <svg className="w-4 h-4 text-gray-500" fill="currentColor" viewBox="0 0 20 20" aria-hidden="true">
      <path
        fillRule="evenodd"
        d="M10 9a3 3 0 100-6 3 3 0 00 6zm-7 9a7 7 0 114 0H3z"
        clipRule="evenodd"
      ></path>
    </svg>
  </div>
)
```

Ini adalah src/components/DefaultAvatar.jsx

```
uppercase">Resources</h3>
    <ul className="text-gray-500 space-y-2">
      <li>
        <a href="https://react.dev" className="hover:text-gray-900 transition-
colors">
          React
        </a>
      </li>
      <li>
        <a href="https://codeigniter.com" className="hover:text-gray-900 transition-
colors">
          CodeIgniter
        </a>
      </li>
      <li>
        <a href="https://tailwindcss.com" className="hover:text-gray-900 transition-
colors">
          Tailwind CSS
        </a>
      </li>
    </ul>
  </div>
</div>
</div>
```

```

        </a>
      </li>
    </ul>
  </div>

  <div>
    <h3 className="mb-4 text-sm font-semibold text-gray-900 uppercase">Follow
us</h3>
    <ul className="text-gray-500 space-y-2">
      <li>
        <a href="#github" className="hover:text-gray-900 transition-colors">
          Github
        </a>
      </li>
      <li>
        <a href="#twitter" className="hover:text-gray-900 transition-colors">
          Twitter
        </a>
      </li>
    </ul>
  </div>
</div>
</div>

  { /* Garis pemisah dan copyright sederhana. */ }
  <hr className="my-6 border-gray-200" />
  <p className="text-center text-sm text-gray-500">© 2026 Blog Posts. All rights
reserved.</p>
</div>
</footer>
)
}

```

// Export default agar Footer bisa dipanggil dari beberapa halaman.
export default Footer

Ini adalah src/components/Footer.jsx

```

// useState mengatur nilai input email/password dan status loading.
import { useState } from 'react'
// useNavigate dipakai untuk redirect setelah login sukses.
import { useNavigate } from 'react-router-dom'
// useAuth menyediakan function login dari AuthContext.
import { useAuth } from '../context/AuthContext'

// Halaman Login meminta email dan password user CI3.
const Login = () => {
  // State email terhubung ke input email.
  const [email, setEmail] = useState('')
  // State password terhubung ke input password.
  const [password, setPassword] = useState('')
  // State error menampilkan pesan gagal login dari backend.
  const [error, setError] = useState('')
  // State loading mencegah submit ganda saat request sedang berjalan.
  const [loading, setLoading] = useState(false)
  // login berasal dari AuthContext dan akan menyimpan token.
  const { login } = useAuth()
  // navigate mengarahkan user ke halaman posts setelah berhasil login.
  const navigate = useNavigate()

  // handleSubmit menjalankan request login ke API CodeIgniter.
  const handleSubmit = async (event) => {
    event.preventDefault()
    setError('')
    setLoading(true)

    try {
      // Function login mengembalikan object success agar halaman bisa menentukan aksi
      berikutnya.
      const result = await login(email, password)

      if (result.success) {
        navigate('/posts')
      }
    }
  }
}

```

```

        } else {
            setError(result.message)
        }
    } catch (err) {
        // Fallback jika terjadi error JavaScript di luar response axios.
        console.error('Login error:', err)
        setError(err.message || 'Terjadi kesalahan saat login. Pastikan server CodeIgniter sudah
berjalan.')
    } finally {
        // Loading selalu dimatikan setelah request selesai.
        setLoading(false)
    }
}

return (
    <div className="min-h-screen bg-gray-100 flex flex-col">
        <div className="flex-1 flex flex-col items-center justify-center px-6 py-8">
            {/* Brand sederhana seperti contoh modul. */}
            <a href="#top" className="flex items-center mb-8 text-2xl font-semibold text-gray-
900">
                <svg className="w-10 h-10 mr-3 text-blue-600" fill="currentColor" viewBox="0 0 20
20" aria-hidden="true">
                    <path d="M5 3a2 2 0 0-2 2v2a2 2 0 02 2h2a2 2 0 02-2v5a2 2 0 00-2-2H5zM5 11a2 2
0 00-2 2v2a2 2 0 02 2h2a2 2 0 02-2v-2a2 2 0 00-2-2H5zM11 5a2 2 0 012-2h2a2 2 0 012 2v2a2 2 0
01-2 2h-2a2 2 0 01-2-2V5zM14 11a1 1 0 011 1v1h1a1 1 0 110 2h-1v1a1 1 0 11-2 0v-1h-1a1 1 0 110-
2h1v-1a1 1 0 011-1z"></path>
                </svg>
                Blog Posts
            </a>

            {/* Card form login. */}
            <div className="w-full max-w-md bg-white rounded-xl shadow-lg border border-gray-200
p-8">
                <h1 className="text-2xl font-bold text-gray-900 mb-6">Sign in to your account</h1>

                {/* Alert hanya muncul saat ada error. */}
                {error && (
                    <div className="mb-6 p-4 text-sm text-red-700 bg-red-100 rounded-lg border border-
red-200" role="alert">
                        {error}
                    </div>
                )}

                <form className="space-y-6" onSubmit={handleSubmit}>
                    {/* Input email wajib diisi karena backend login mencari user berdasarkan email.
*/}
                    <div>
                        <label htmlFor="email" className="block mb-2 text-sm font-medium text-gray-900">
                            Your email
                        </label>
                        <input
                            type="email"
                            name="email"
                            id="email"
                            autoComplete="email"
                            value={email}
                            onChange={(event) => setEmail(event.target.value)}
                            className="w-full px-4 py-3 bg-gray-50 border border-gray-300 rounded-lg text-
gray-900 placeholder-gray-400 focus:ring-2 focus:ring-blue-500 focus:border-blue-500 outline-
none transition-all"
                            placeholder="name@company.com"
                            required
                        />
                    </div>

                    {/* Input password akan dikirim ke endpoint api/auth/login. */}
                    <div>
                        <label htmlFor="password" className="block mb-2 text-sm font-medium text-gray-
900">
                            Password
                        </label>
                        <input
                            type="password"
                            name="password"

```

```

        id="password"
        autoComplete="current-password"
        value={password}
        onChange={ (event) => setPassword(event.target.value)}
        placeholder="Masukkan password"
        className="w-full px-4 py-3 bg-gray-50 border border-gray-300 rounded-lg text-
gray-900 placeholder-gray-400 focus:ring-2 focus:ring-blue-500 focus:border-blue-500 outline-
none transition-all"
        required
      />
    </div>

    { /* Opsi remember me dibuat mengikuti tampilan modul. */ }
    <div className="flex items-center justify-between">
      <div className="flex items-center">
        <input id="remember" type="checkbox" className="w-4 h-4 border border-gray-300
rounded bg-gray-50 focus:ring-2 focus:ring-blue-500" />
        <label htmlFor="remember" className="ml-2 text-sm text-gray-600">
          Remember me
        </label>
      </div>
      <a href="#forgot-password" className="text-sm font-medium text-blue-600
hover:text-blue-500">
        Forgot password(c)
      </a>
    </div>

    { /* Tombol submit berubah teks saat proses login sedang berlangsung. */ }
    <button
      type="submit"
      disabled={loading}
      className="w-full py-3 px-4 bg-blue-600 hover:bg-blue-700 text-white font-medium
rounded-lg transition-colors focus:ring-4 focus:ring-blue-300 disabled:opacity-50
disabled:cursor-not-allowed"
    >
      { loading (c) 'Signing in...' : 'Sign in' }
    </button>
  </form>
</div>
</div>

  { /* Footer kecil khusus halaman login. */ }
  <footer className="bg-white border-t border-gray-200 py-4">
    <p className="text-center text-sm text-gray-500">© 2026 Blog Posts. All rights
reserved.</p>
  </footer>
</div>
)
}

```

// Export default agar route /login dapat merender halaman ini.
export default Login

Ini adalah src/pages/Login.jsx

```

/* eslint-disable react-hooks/set-state-in-effect */
// useCallbak menjaga function fetch stabil untuk useEffect, useEffect menjalankan fetch saat
halaman dibuka.
import { useCallback, useEffect, useState } from 'react'
// postsAPI berisi request ke endpoint api/post pada backend CI3.
import { postsAPI } from '../services/api'
// Navbar tampil pada bagian atas halaman.
import Navbar from '../components/Navbar'
// Footer tampil pada bagian bawah halaman.
import Footer from '../components/Footer'
// PostCard merender setiap item post.
import PostCard from '../components/PostCard'

// PostList menampilkan semua artikel yang dikirim backend.
const PostList = () => {
  // State posts menyimpan array hasil response API.
  const [posts, setPosts] = useState([])
  // Loading dipakai untuk menampilkan spinner saat data belum datang.
  const [loading, setLoading] = useState(true)

```

```

// Error dipakai untuk menampilkan pesan gagal fetch.
const [error, setError] = useState('')

// fetchPosts mengambil daftar post dari endpoint api/post.
const fetchPosts = useCallback(async () => {
  try {
    setLoading(true)
    // Backend CI3 mengirim data pada response.data.data.
    const response = await postsAPI.getAll()
    setPosts(response.data(c).data || [])
    setError('')
  } catch (err) {
    // Pesan error dibuat jelas untuk user praktikum.
    setError('Gagal memuat posts. Pastikan XAMPP dan API CodeIgniter Pekan-09 sudah
berjalan.')
    console.error('Error fetching posts:', err)
  } finally {
    // Loading dimatikan setelah request selesai.
    setLoading(false)
  }
}, [])

// Effect ini menjalankan fetchPosts sekali saat komponen mount.
useEffect(() => {
  fetchPosts()
}, [fetchPosts])

return (
  <div className="min-h-screen bg-gray-100 flex flex-col">
    <Navbar />
    <main className="flex-1">
      <div className="py-8 px-4 mx-auto max-w-screen-xl lg:py-16 lg:px-6">
        /* Judul halaman seperti contoh output modul. */
        <div className="mx-auto max-w-screen-sm text-center lg:mb-16 mb-8">
          <h2 className="mb-4 text-3xl lg:text-4xl tracking-tight font-extrabold text-gray-
900">Our Blog</h2>
          <p className="font-light text-gray-500 sm:text-xl">
            Explore our collection of articles from the CodeIgniter 3 REST API.
          </p>
        </div>

        /* Kondisi loading, error, kosong, dan sukses dipisahkan agar UI mudah dipahami.
*/
        {loading (c) (
          <div className="flex justify-center items-center py-20">
            <div className="animate-spin rounded-full h-12 w-12 border-b-2 border-blue-
600"></div>
          </div>
        ) : error (c) (
          <div className="text-center py-20">
            <p className="text-red-500 mb-4">{error}</p>
            <button
              type="button"
              onClick={fetchPosts}
              className="text-white bg-blue-600 hover:bg-blue-700 font-medium rounded-lg
text-sm px-5 py-2.5"
            >
              Coba Lagi
            </button>
          </div>
        ) : posts.length === 0 (c) (
          <div className="text-center py-20">
            <p className="text-gray-500">Belum ada posts.</p>
          </div>
        ) : (
          <div className="grid gap-8 lg:grid-cols-2">
            {posts.map((post) => (
              <PostCard key={post.id} post={post} />
            ))}
          </div>
        )}
      </div>
    </main>
    <Footer />
  </div>
)

```



```

    </div>
  )
}

```

```

// Export default agar route /posts dapat merender halaman ini.
export default PostList

```

Ini adalah src/pages/PostList.jsx

```

/* eslint-disable react-hooks/set-state-in-effect */
// useCallback menjaga function fetch stabil, useEffect mengambil data saat ID berubah.
import { useCallback, useEffect, useState } from 'react'
// useParams membaca ID dari URL, Link membuat tombol kembali tanpa reload.
import { Link, useParams } from 'react-router-dom'
// postsAPI berisi request detail post ke backend CI3.
import { postsAPI } from '../services/api'
// Navbar digunakan di bagian atas halaman.
import Navbar from '../components/Navbar'
// Footer digunakan di bagian bawah halaman.
import Footer from '../components/Footer'
// DefaultAvatar menampilkan ikon author.
import DefaultAvatar from '../components/DefaultAvatar'

// PostDetail menampilkan satu artikel berdasarkan ID route.
const PostDetail = () => {
  // ID berasal dari route /posts/:id.
  const { id } = useParams()
  // State post menyimpan object detail dari response API.
  const [post, setPost] = useState(null)
  // Loading menampilkan spinner saat request berjalan.
  const [loading, setLoading] = useState(true)
  // Error menampilkan pesan jika post tidak ditemukan atau server gagal.
  const [error, setError] = useState('')

  // fetchPost mengambil satu post dari endpoint api/post/:id.
  const fetchPost = useCallback(async () => {
    try {
      setLoading(true)
      // Backend CI3 mengirim detail pada response.data.data.
      const response = await postsAPI.getById(id)
      setPost(response.data(c).data || null)
      setError('')
    } catch (err) {
      // Status 404 diberi pesan khusus agar user tahu ID tidak ada.
      if (err.response(c).status === 404) {
        setError('Post tidak ditemukan.')
      } else {
        setError('Gagal memuat post. Pastikan API CodeIgniter Pekan-09 sudah berjalan.')
      }
      console.error('Error fetching post:', err)
    } finally {
      // Loading selalu dimatikan setelah request selesai.
      setLoading(false)
    }
  }, [id])

  // Fetch detail dijalankan setiap ID berubah.
  useEffect(() => {
    fetchPost()
  }, [fetchPost])

  // formatDate mengubah created_at menjadi format tanggal Indonesia.
  const formatDate = (dateString) => {
    if (!dateString) return ''

    // Date dibuat dari string timestamp database.
    const date = new Date(dateString)
    if (Number.isNaN(date.getTime())) return ''

    // Format Indonesia membuat tanggal lebih mudah dibaca.
    return date.toLocaleDateString('id-ID', {
      day: 'numeric',
      month: 'long',

```

```

        year: 'numeric',
    })
}

// Tampilan loading awal saat detail post sedang dimuat.
if (loading) {
    return (
        <div className="min-h-screen bg-gray-100 flex items-center justify-center">
            <div className="animate-spin rounded-full h-12 w-12 border-b-2 border-blue-600"></div>
        </div>
    )
}

// Tampilan error tetap membawa Navbar dan Footer agar layout konsisten.
if (error) {
    return (
        <div className="min-h-screen bg-gray-100 flex flex-col">
            <Navbar />
            <div className="flex-1 flex items-center justify-center py-8 px-4 mx-auto max-w-screen-xl lg:py-16 lg:px-6">
                <div className="text-center py-20">
                    <p className="text-red-500 mb-4">{error}</p>
                    <Link to="/posts" className="text-white bg-blue-600 hover:bg-blue-700 font-medium rounded-lg text-sm px-5 py-2.5">
                        Kembali ke Daftar Posts
                    </Link>
                </div>
            </div>
            <Footer />
        </div>
    )
}

return (
    <div className="min-h-screen bg-gray-100 flex flex-col">
        <Navbar />
        <main className="flex-1">
            <div className="py-8 px-4 mx-auto max-w-screen-lg lg:py-16 lg:px-6">
                <!-- Link kembali ke halaman daftar post. -->
                <div className="mb-6">
                    <Link to="/posts" className="inline-flex items-center text-sm font-medium text-blue-600 hover:text-blue-500 transition-colors">
                        <svg className="mr-2 w-4 h-4" fill="currentColor" viewBox="0 0 20 20" aria-hidden="true">
                            <path
                                fillRule="evenodd"
                                d="M9.707 16.707a1 1 0 01-1.414 1.414l-6a1 1 0 01-1.414 1.414L5.414 9H17a1 1 0 10 2 0H5.414L4.293 4.293a1 1 0 010 1.414z"
                                clipRule="evenodd"
                            ></path>
                        </svg>
                        Back to Posts
                    </Link>
                </div>

                <!-- Article detail menampilkan data lengkap dari field title, author, article, created_at. -->
                <article className="bg-white rounded-lg border border-gray-200 shadow-md overflow-hidden">
                    <div className="p-6 lg:p-10">
                        <div className="flex flex-wrap items-center gap-4 mb-6">
                            <span className="bg-blue-100 text-blue-800 text-xs font-medium inline-flex items-center px-2.5 py-0.5 rounded">
                                <svg className="mr-1 w-3 h-3" fill="currentColor" viewBox="0 0 20 20" aria-hidden="true">
                                    <path
                                        fillRule="evenodd"
                                        d="M2 5a2 2 0 012-2h8a2 2 0 012 2v10a2 2 0 012 2h4a2 2 0 01-2-2V5zm3 1h6v4H5V6zm6 6H5v2h6v-2z"
                                        clipRule="evenodd"
                                    ></path>
                                </svg>
                                Article

```

```

    </span>

    <div className="flex items-center space-x-2">
      <DefaultAvatar />
      <span className="font-medium text-sm text-gray-700">{post(c).author}</span>
    </div>

    {post(c).created_at && (
      <span className="text-sm text-gray-500 flex items-center">
        <svg className="w-4 h-4 mr-1" fill="currentColor" viewBox="0 0 20 20"
aria-hidden="true">
          <path
            fillRule="evenodd"
            d="M6 2a1 1 0 0-1 1v1H4a2 2 0 0-2 2v10a2 2 0 002 2h12a2 2 0 002-
2V6a2 2 0 00-2-2h-1V3a1 1 0 10-2 0v1H7V3a1 1 0 00-1-1zm0 5a1 1 0 00 2h8a1 1 0 100-2H6z"
            clipRule="evenodd"
          ></path>
        </svg>
        {formatDate(post.created_at)}
      </span>
    )}
  </div>

  <h1 className="mb-6 text-3xl lg:text-4xl font-bold tracking-tight text-gray-
900">{post(c).title}</h1>

  {/** Artikel dipecah per baris agar newline dari database tetap terbaca sebagai
paragraf. */}
  <div className="text-gray-600 leading-relaxed text-lg">
    {(post(c).article || '').split('\n').map((paragraph, index) => (
      <p key={index} className="mb-4">
        {paragraph}
      </p>
    ))}
  </div>
</div>
</article>
</div>
</main>
<Footer />
</div>
)
}

```

```

// Export default agar route /posts/:id dapat merender halaman ini.
export default PostDetail

```

Ini adalah src/pages/PostDetail.jsx

```

/** Mengaktifkan reset style dasar Tailwind. */
@tailwind base;

/** Mengaktifkan class komponen Tailwind bila nanti ditambahkan. */
@tailwind components;

/** Mengaktifkan utility class Tailwind seperti bg-gray-100, px-4, dan grid. */
@tailwind utilities;

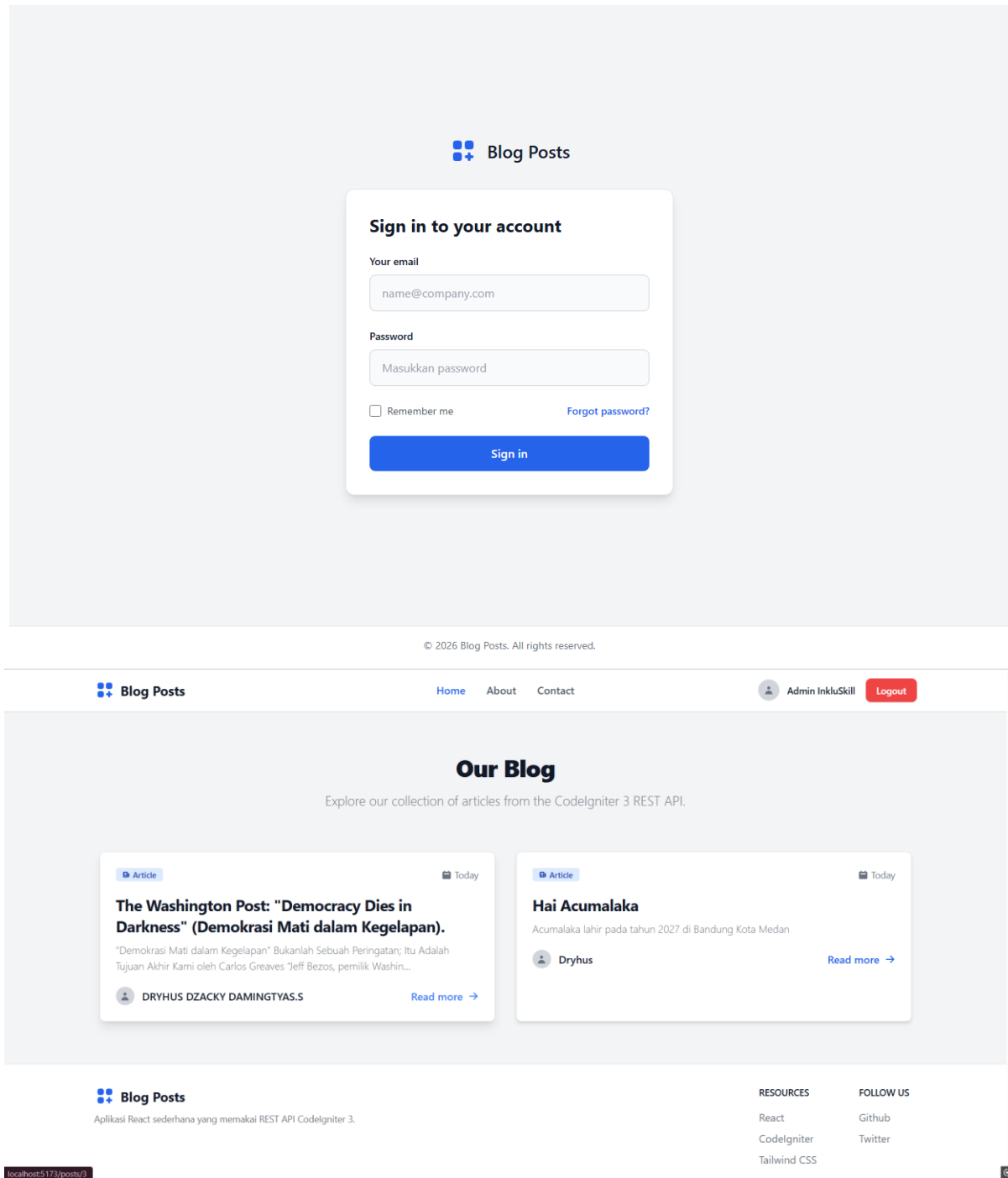
/** Root disiapkan sesuai modul agar browser memahami dukungan mode terang/gelap. */
:root {
  color-scheme: light;
}

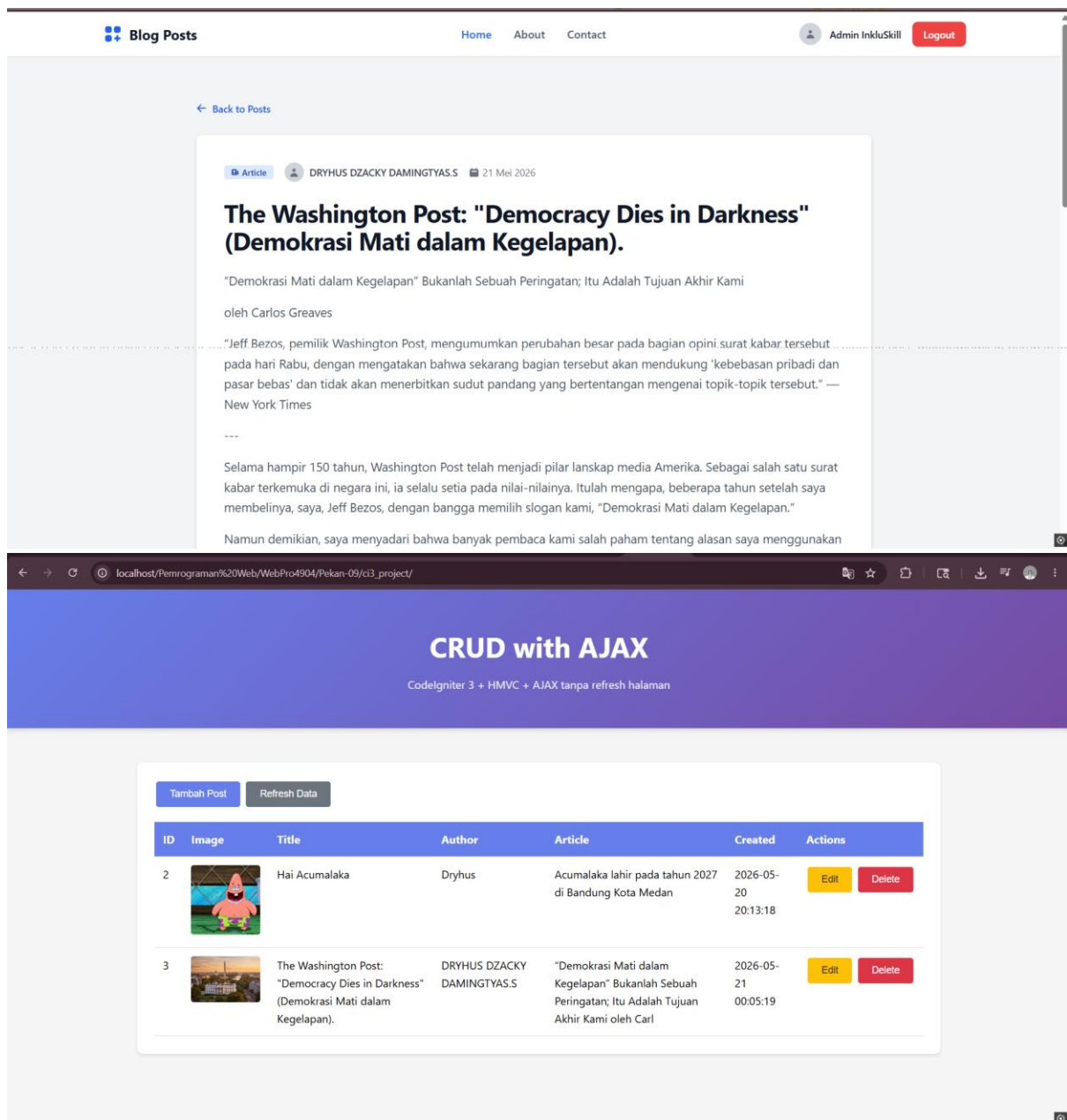
/** Font global dibuat mirip Inter agar tampilan tetap rapi tanpa dependency font tambahan. */
body {
  margin: 0;
  font-family: 'Inter', -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto, sans-serif;
}

/** Box sizing global mencegah padding membuat layout melebar dari layar. */
* {
  box-sizing: border-box;
}

```

}
Ini adalah src/index.css





Ini adalah output programnya pada halaman login aplikasi Blog Posts.

E. Analisis dan Pembahasan

Pada praktikum Pekan 12 ini dibuat aplikasi React berbasis Vite yang terhubung dengan REST API CodeIgniter 3. Aplikasi ini tidak hanya menampilkan halaman statis, tetapi sudah memiliki alur autentikasi, penyimpanan token, route yang dilindungi, serta pengambilan data post dari backend.

File `App.jsx` berfungsi sebagai pusat routing aplikasi. Route `/login` digunakan untuk halaman login, sedangkan route `/posts` dan `/posts/:id` dibungkus dengan `ProtectedRoute`. Dengan struktur ini, user yang belum memiliki token tidak dapat langsung membuka halaman daftar post maupun detail post.

Pengelolaan autentikasi dilakukan pada `AuthContext.jsx`. Context ini menyimpan user, token, status loading, fungsi login, dan fungsi logout. Token dan data user disimpan di localStorage agar sesi tetap aktif walaupun browser direfresh. Saat aplikasi dibuka, token

lama divalidasi melalui endpoint ``auth/me`` sehingga token yang rusak atau expired dapat dibersihkan.

File ``api.js`` berperan sebagai pusat komunikasi dengan backend. Axios instance dibuat dengan base URL menuju project CodeIgniter 3 Pekan-09. Interceptor request menambahkan header Authorization Bearer ketika token tersedia, sedangkan interceptor response menangani response 401 dengan menghapus token dan mengarahkan user ke halaman login.

Halaman ``Login.jsx`` menggunakan controlled component untuk input email dan password. Ketika form dikirim, fungsi login dari AuthContext dipanggil. Jika login berhasil, user diarahkan ke halaman ``/posts``. Jika gagal, pesan error dari backend ditampilkan pada halaman login.

Halaman ``PostList.jsx`` mengambil data artikel melalui ``postsAPI.getAll()``. Data yang berhasil diterima disimpan dalam state ``posts``, kemudian ditampilkan menggunakan komponen ``PostCard``. Halaman ini juga memiliki kondisi loading, error, dan empty state sehingga user mendapatkan informasi yang jelas terhadap kondisi data.

Komponen ``PostCard.jsx`` bertugas menampilkan ringkasan artikel. Setiap card menampilkan kategori artikel, tanggal, judul, potongan isi artikel, author, dan tombol Read more. Tombol tersebut menggunakan React Router Link sehingga perpindahan ke halaman detail terjadi tanpa reload halaman.

Halaman ``PostDetail.jsx`` membaca parameter ID dari URL menggunakan ``useParams``. ID tersebut digunakan untuk mengambil data detail artikel melalui endpoint ``api/post/:id``. Jika data berhasil didapatkan, halaman menampilkan judul, author, tanggal, dan isi artikel. Jika post tidak ditemukan, halaman menampilkan pesan error dan tombol kembali ke daftar post. Tampilan aplikasi dibuat menggunakan Tailwind CSS. Class Tailwind dipakai untuk layout, warna, spacing, typography, card, tombol, grid, navbar, dan footer. Dengan Tailwind, tampilan menjadi lebih konsisten, responsif, dan mudah dikembangkan tanpa membuat banyak file CSS manual.

Secara keseluruhan, praktikum ini melatih integrasi frontend React dengan backend REST API, penggunaan autentikasi token, pengaturan route, manajemen state global menggunakan context, serta penyajian data dinamis dari server ke antarmuka web.

F. Kesimpulan

Berdasarkan praktikum yang telah dilakukan, aplikasi Blog Posts berhasil dibuat menggunakan React JS, Vite, React Router DOM, Axios, dan Tailwind CSS. Aplikasi ini mampu menampilkan halaman login, menyimpan token autentikasi, melindungi halaman post, menampilkan daftar post, membuka detail post, dan melakukan logout.

Praktikum ini memberikan pemahaman tentang cara membangun aplikasi frontend modern yang terhubung dengan backend REST API. Mahasiswa juga dapat memahami pentingnya AuthContext, ProtectedRoute, interceptor axios, localStorage, dan routing dalam membuat aplikasi web yang memiliki alur autentikasi dan data dinamis.

LINK REPOSITORY GITHUB DRYHUS

<https://github.com/AriqAhmadNayaka/WebPro4904/tree/DryhusDzackyDamingtyas.S-707012530010>